Sardar Vallabhbhai National Institute of Technology
Surat - 395007
Department of Artificial Intelligence
Probabilistic Graphical Models (AI453)

[Lab Practical - 6]

Problem Statement: Design and Implementation of **Linear-Time Conditional Independence Algorithms — Bayes-Ball Reachability and the Markov Blanket** — for Directed Graphical Models.

INTRODUCTION

Conditional independence in a Bayesian network is a property of the graph alone. Two nodes X and Y are d-separated by an evidence set Z when every trail between them is blocked, and d-separation implies conditional independence in every distribution that factorises over the graph. Reading independences off the structure, without touching a single probability, is what makes inference on large networks possible at all.

The definition is stated over trails, and implementing it literally means enumerating them. The number of trails between two nodes grows exponentially with the size of the graph, so the direct approach fails on networks of even modest size.

Key Insight: The same question is answered by a single traversal if the search state is a **(node, direction)** pair rather than a node. A ball arriving at a node from a child behaves differently from one arriving from a parent, and this asymmetry is precisely what separates d-separation from ordinary graph connectivity. The traversal visits at most 2n states and traverses each edge at most twice, giving O(n + m) per query regardless of how many trails the graph contains.

A second structural object follows from the same traversal. The Markov blanket of a node — its parents, its children, and the other parents of those children — is the smallest set that renders the node independent of everything else in the network. It is the quantity every Gibbs sampler needs at every step.

TEST NETWORKS

File	Nodes	Edges	Queries	Description
`case_p6_sample.in`	5	4	12	Student network of Lab 4; answers must match Lab 4 exactly
`case_p6_student.in`	8	14	12	Network fitted from real student data in Practical 5
`case_p6_main.in`	15	22	40	Evaluation network; submit answers for this file
`case_p6_wide.in`	91	120	7	2^30 directed paths between the two endpoints

INPUT AND OUTPUT SPECIFICATION

Nodes are numbered 0 to n−1. Every record occupies its own line; read the file line by line.

```
n m                  n nodes, m edges
u v                  m lines, one per directed edge u -> v
q                    number of queries
```

```
x y z1 z2 ... zk       q lines: the pair, then the evidence set Z
```

In every query x, y and the nodes of Z are distinct. The evidence set may be empty. The program takes the input filename as its first command-line argument and writes to standard output. Two modes are required:

```
python3 lab6.py case_p6_main.in              -> q lines of YES / NO
python3 lab6.py case_p6_main.in --blanket    -> n lines, one blanket each
```

Default mode: print YES if x and y are d-separated by Z, otherwise NO, one line per query in input order. Capitals only, nothing else on the line, no headings and no blank lines.

Blanket mode: print n lines; line i holds the Markov blanket of node i as node numbers in increasing order separated by single spaces. An empty blanket prints an empty line.

Constraints: Python 3 standard library only. `networkx` and `pgmpy` implement d-separation directly and are not permitted. Do not hard-code any filename or any size.

Problem Statement 1: Graph Construction and Ancestor Computation

Parse the input file into parent and child adjacency structures, and compute the ancestor closure of the evidence set.

Why the ancestor set is needed:

- A collider admits the ball only when the collider itself, or one of its descendants, lies in Z
- Equivalently, the collider must be an ancestor of Z, or a member of Z
- A single upward sweep from Z yields every such node at once, so the test becomes a set membership check

Tasks:

a) Implement `read_graph(path)` returning n, parents, children and the query list

b) Implement `ancestors_of(parents, Z)` by iterative upward traversal; avoid recursion so deep graphs do not overflow the stack

c) Verify on `case_p6_sample.in` that the ancestors of {3} are {0, 1, 2, 3}, and state why a node is counted as its own ancestor

Problem Statement 2: Bayes-Ball Reachability Algorithm [25 Marks]

Implement reachability over (node, direction) pairs. Write up for a ball that arrived from a child, moving against the arrows, and down for one that arrived from a parent.

Traversal rules:

State	Condition	Pass the ball to
(v, up)	v not in Z	every parent as up, every child as down
(v, up)	v in Z	nothing; the trail is blocked
(v, down)	v not in Z	every child as down
(v, down)	v in Z or v in ancestors(Z)	every parent as up (collider opened by evidence)

Tasks:

d) Implement `reachable(parents, children, x, Z)` starting from (x, up) and returning the set of nodes not d-separated from x by Z

e) Maintain the visited set over **pairs**, never over nodes. Explain in a comment which trails are lost if nodes are used, and construct a graph on which that error changes an answer

f) Collect a node into the result only when it is not itself a member of Z, and justify the exclusion

g) Confirm that a node may legitimately be visited twice, once in each direction, and identify such a node in `case_p6_student.in`

Problem Statement 3: d-separation Queries and Cross-Validation

Answer the queries and validate the implementation against the trail-enumeration program written in Lab 4.

Tasks:

h) Decide each query: x and y are d-separated by Z exactly when y is absent from `reachable(x, Z)`

i) Run on `case_p6_sample.in`, which is the Lab 4 sample file, and verify that `diff lab4_output.txt lab6_output_sample.txt` prints nothing

j) Where the two programs disagree, determine which is wrong by working the query out on paper, and record the diagnosis

k) Produce `lab6_output.txt` for `case_p6_main.in`

Problem Statement 4: Complexity Analysis

Demonstrate empirically that the cost of the traversal is independent of the number of trails.

Tasks:

l) Instrument the search with a counter for the number of (node, direction) pairs popped, printed to `stderr` so that standard output remains clean

m) Tabulate n, m, 2n and the observed pair count for all four test networks

n) Run on `case_p6_wide.in`, which holds $2^{30} \approx 1.07 \times 10^9$ directed paths between its endpoints, and record the wall-clock time

o) Start the Lab 4 program on the same file, record how long you allowed it before abandoning it, and state the bound on the pair count in terms of n and m, explaining why the number of trails does not appear in that bound

Problem Statement 5: Markov Blanket Computation [15 Marks]

Compute the Markov blanket of every node.

The blanket of a node v consists of:

- the parents of v
- the children of v
- the other parents of the children of v, that is, the co-parents

Tasks:

p) Implement `blanket(parents, children, v)` returning a sorted list without duplicates, never containing v itself

q) Implement the `--blanket` output mode to the specification above

r) Report the blanket size of every node in `case_p6_main.in` and identify the largest and smallest

s) State the relation between a node with no children and the size of its blanket

Problem Statement 6: Structural Verification of the Blanket

Establish, using your own reachability code, that the blanket separates.

Tasks:

t) For every node v and every node w that is neither v nor in blanket(v), verify that v and w are d-separated by blanket(v). Run over all four networks and report only the failures; there must be none

u) Delete the co-parent term from the blanket, repeat the check, and report the number of failures introduced

v) Identify the query in `case_p6_sample.in` that exhibits this failure, and explain in terms of colliders why the co-parents cannot be omitted

Problem Statement 7: Numerical Verification on Real Data

Requires `lab5_bn.py` and `student-por.csv` from Practical 5. `case_p6_student.in` encodes that network as 0 medu, 1 goout, 2 fail, 3 higher, 4 study, 5 absent, 6 g1, 7 grade.

Tasks:

w) Print every blanket of the Practical 5 network in variable names rather than indices, and comment on which node has the smallest blanket and why

x) Using the fitted CPTs and the `infer()` routine of Practical 5, compute P(g1 = 1 | blanket of g1 all set to 1), then recompute with medu = 1 appended to the evidence. Since medu lies outside the blanket the two values must agree to at least ten decimal places

y) Repeat for two further nodes of your choice and tabulate the results

z) State how much of a network of ten thousand nodes must be examined in order to resample one variable, and name the sampling algorithm that depends on this fact

SUBMISSION

Submit exactly two files, with these names:

```
lab6.py            all of your code, in this one file
lab6_output.txt    the lines your program printed for case_p6_main.in
```

The program will be executed on an input file you have not seen, possibly in a different directory and with a different number of nodes and queries.

REFERENCES

1. D. Koller and N. Friedman, Probabilistic Graphical Models: Principles and Techniques, MIT Press, Chapter 3.

2. R. D. Shachter, Bayes-Ball: The Rational Pastime, Uncertainty in Artificial Intelligence, 1998.

3. A. Darwiche, Modeling and Reasoning with Bayesian Networks, Chapter 4.

4. P. Cortez and A. Silva, Using Data Mining to Predict Secondary School Student Performance, 2008. UCI Machine Learning Repository.